

Final Paper SEP CV & DL

Jan Sternberg Ivan Kazakov Ming Gui Mark Koester

{Jan.Sternberg, Kazakov.Ivan, Huang.Ziyang, Mark.Koester}@campus.lmu.de

Abstract

We will write this as last thing.

1. Introduction

1.1. Animal Localization and Preprocessing

In this study, subject localization and extraction are performed using a specialized two-stage pipeline designed to isolate the primary animal before fine-grained classification. To comply with the strict inference time constraints of the project—averaging under 5 seconds per image—while maintaining high spatial accuracy, we employ an off-the-shelf YOLOv6 object detector (Li et al., 2022a). Specifically, the lightweight YOLOv6-S architecture is utilized. The YOLOv6 framework introduces significant advancements in single-stage detection, utilizing hardware-aware neural network design and re-parameterization techniques to maximize throughput without sacrificing performance (Li et al., 2022b). Furthermore, it incorporates an adaptive self-distillation strategy during training, which dynamically adjusts knowledge transfer, rendering it highly robust for real-time edge detection tasks (Zhang et al., 2020). To ensure robustness against background interference, our pipeline processes raw RGB images directly, leveraging the detector’s native Non-Maximum Suppression (NMS) capabilities. Upon inference, the network outputs precise bounding box coordinates. Our algorithm explicitly filters these predictions for the target species (cats and dogs, corresponding to COCO IDs 15 and 16). In scenarios where multiple target animals are present within a single frame, the bounding box with the largest spatial area is identified as the primary subject, directly adhering to the dataset’s ground-truth definition. A critical challenge in fine-grained classification is contextual confounding, where secondary animals in the background cause erroneous predictions. To mitigate this, we propose and implement a zero-out masking technique: all bounding boxes corresponding to non-primary detections within the frame are explicitly blacked out (pixel values set to zero). The primary animal is subsequently cropped and bilinearly resized to a standardized dimension of 224×224 pixels,

meeting the strict input requirements for subsequent fine-grained classification networks. Images yielding no valid detections—indicating the presence of confounders or the absence of target species—are routed to a structural fallback mechanism. This mechanism passes a resized version of the original frame forward while assigning a ‘neither’ label, forcing the pipeline to reliably return the required -1 reject class.

2. Detector

2.1. Detector Selection and Implementation

To optimize localization speed and accuracy, we initially evaluated Google SpeciesNet [1], an ensemble architecture highly regarded for its robust handling of challenging environments. SpeciesNet utilizes the MegaDetector framework [3] and boasts an impressive 98.6% empty image exclusion accuracy, effectively filtering out blank “wind-blows” images. However, while the system is highly optimized for batch processing of static, low-quality, poorly lit, or camouflaged trail-camera photographs, empirical testing revealed that its heavy computational overhead consistently exceeded our expectations and the strict 5-second overall inference constraint dictated by the project specifications. Consequently, as outlined in our preliminary report, we proceeded with YOLOv6, a lightweight and highly efficient single-stage object detector [2].

The selected YOLOv6 model is a standardized architecture pretrained on the comprehensive Common Objects in Context (COCO) dataset. This pretraining allowed us to directly leverage established COCO class identifiers (numeric IDs assigned to different object categories worldwide) to explicitly filter for our target species: cats (class ID 15) and dogs (class ID 16). If the detector identifies neither class, the pipeline immediately forwards the rejection class (-1), efficiently bypassing the downstream prediction network. Preliminary inference tests on our dataset demonstrated exceptional localization capabilities, achieving a 100% accuracy rate in correctly identifying the primary subject, which validated this approach for our pipeline.

A significant concern during initial testing was the potential for grouped or closely situated animals to be merged into a single bounding box. This would severely degrade

the performance of the subsequent fine-grained classifier, which we trained from scratch, as it would struggle to determine the correct species. However, this concern was scattered during testing with multiple complex pictures, including zoo animals. We observed that YOLOv6 effectively avoids this issue by creating a dedicated bounding box for each individual animal. To strictly adhere to the project's evaluation rules, our algorithm compares the spatial dimensions of all detected bounding boxes and forwards only the largest one.

Furthermore, to ease the representational burden on our prediction model and to eliminate the traits of other animals that may overlap with the forwarded bounding box of the searched animal, we implemented a zero-out masking strategy. In scenarios where smaller animals overlap with the primary subject, the overlapping regions are explicitly blacked out. We see a distinct advantage in time and accuracy by eliminating the traits of the smaller animals. However, we are aware of the inherent trade-off: this masking strategy could temporarily make it harder for our model to detect the primary animal, as a large chunk of its visual information might go missing.

3. Pipeline

we describe our pipeline

4. Detection Model

Here we write down our way code our detection model

4.1. Training our Model

Here we write down our way of training the chosen model.

5. Training

During training with a small subset of data, we realized that we heavily differ from the original 100% accuracy of the detector, as our dataset of 20 animals per breed was too small. Even though the Cat-API, from which we pulled our data, has a large dataset, the data is not properly labeled, and we could only obtain around 12 pictures per breed. As hand-labeling another eight pictures to reach the target of 20 pictures per breed takes too long, we avoided going forward with this and took our chance with further officially provided datasets.

Here, we implemented the Oxford Dataset, which increased our pool by 2,744 pictures of the animal breeds we needed. However, upon checking, we realized that *Tabby* & *Tiger.cat* are very similar, and *Tabby* is not a breed but a fur pattern. In this case, we proceeded with *Tiger.cat*. Down the line, we recognized that we had too few pictures of *Golden_Retriever*, *German_Shepherd*, *Siberian_Husky*, *Dalmatian*, and *Rottweiler*, as these are not included in the

Oxford Dataset. Due to this, we went further and implemented the Stanford Dogs Dataset, which has 150 pictures per specified breed and helped us to finalize our training dataset. As we still struggled with tabby and tiger cat, we proceeded just for these breeds to pull 30 Pictures from Unsplash.com and Pixabay.com. As we were still unsatisfied with the training results especially for these breeds, we increased the density to 50 pictures per breed, which pushed the performance over the 80% accuracy, which we held as our main goal.

During training we realized that our algorithm was still getting worse from time to time, due to this we verified our picture dataset and realized that dalmatian have been excluded for now of the dataset. As none of the api connections we used earlier had any dalmatian pictures, we downloaded a file of 359 Dalmatian pictures and included this in our dataset. Through this after 10 Epochs of training we had a big jump from 17 to 69% accuracy.

6. ivan

Blank text Describe your progress here

sources like this!

@article{Alpher02, author = FirstName Alpher, title = Frobnication, journal = PAMI, volume = 12, number = 1, pages = 234–778, year = 2002

References

- [1] Google Research and Wildlife Insights. Speciesnet: An open-source image classifier for motion-triggered wildlife cameras. Available via Kaggle Models and GitHub, 2025. [1](#)
- [2] Chuyi Li, Lulu Li, Hongliang Jiang, Kaihui Weng, Yifei Geng, Liang Li, Zaidan Ke, Qingyuan Li, Meng Cheng, Weiqiang Nie, and Yandong Li. Yolov6: A single-stage object detection framework for industrial applications. *arXiv preprint arXiv:2209.02976*, 2022. [1](#)
- [3] Microsoft AI for Good Lab. Megadetector v6: Open-source camera-trap ai. GitHub repository, 2026. [1](#)